COMPREHENSIVE SYSTEMS & ARCHITECTURE GUIDE

CAMPUS PRINT PLATFORM

Product Architecture, Workflows, and Deep Dive Documentation

SECTION 1: PROJECT OVERVIEW

What problem Campus Print solves

In traditional university and college environments, the process of printing documents is highly inefficient. Students frequently experience long wait times at campus print shops, deal with the hassle of transferring files via physical USB drives or emailing them to public computers (which introduces severe security and privacy risks), and face delays caused by manual payment processing and order management by shop owners.

Campus Print solves this by fully digitizing the print queue and order management system. It removes the need for physical presence during the order placement and printing phase, eliminating queues and hardware sharing.

Why the project was built

The project was built to modernize campus infrastructure and introduce a seamless, contactless, and efficient printing workflow. It bridges the gap between students who need quick prints before classes and print shop owners who struggle with managing peak-hour rushes, disorganized email queues, and manual payment verification.

Target users

1. **Students & Faculty:** Users who need to print documents securely and quickly without waiting in line.

2. **Campus Print Shop Owners/Operators:** Small business owners or university staff managing printing facilities, seeking a streamlined way to

handle incoming print jobs, track revenue, and automate the printer hardware.

Business value

- **Operational Efficiency:** Automates the print spooling process, reducing the time spent per customer from minutes to seconds.
- **Increased Revenue:** By reducing friction and wait times, shops can process a significantly higher volume of print jobs per hour.
- **Cost Reduction:** Eliminates the need for multiple public-facing PCs for students to log into just to print.
- **Data Security:** Students no longer need to log into personal email accounts or plug USBs into infected public computers.

Current MVP features

- Real-time document upload portal with PDF page parsing.
- Dynamic fare estimation based on page count and copies.
- Live real-time queue tracking via Server-Sent Events (SSE).
- Centralized Admin Portal for print operators to monitor the queue.
- Automated Windows Print Spooler integration (Print Daemon) for zero-click physical printing.
- Cloudflare Tunnel integration for secure local-to-public network routing.

Future scalability opportunities

- Multi-shop onboarding (SaaS model for multiple campus shops).
- Integration with UPI/Stripe for automated digital payments.
- Print credits system for faculty/staff allocations.
- Mobile application with push notifications for print completion.

Traditional Process vs Campus Print Process

Traditional Process

Student visits shop

↓

Waits in queue

↓

Transfers files manually (USB/Email)

↓

Operator calculates cost manually

↓

Makes payment

↓

Operator initiates print

↓

Receives print

Campus Print Process

Upload document remotely

↓

System automatically calculates cost & pages

↓

Store receives order instantly in queue

↓

System prints automatically (Print Daemon)

↓

Student notified and collects print

SECTION 2: BUSINESS WORKFLOW

The business workflow of Campus Print is designed for a frictionless, non-blocking asynchronous user journey.

Student Journey

1. **Access Portal:** Student accesses the unique Cloudflare-routed URL from their mobile device or laptop.

2. **Upload Document:** Student drops a PDF/Image into the portal. The frontend extracts page counts.

3. **Configure Print:** Student selects copies, simplex/duplex, and specific page ranges.

4. **Submit:** Document uploads to the backend. Student receives a unique Print Token (e.g., PRNT-123).

5. **Track:** Student watches the live queue tracker which updates in real-time as the physical printer processes jobs ahead of them.

6. **Collect:** Student visits the shop, presents the token, and collects the printed document.

Store Owner Journey

1. **Boot System:** Owner starts the Node.js backend and the Print Client Daemon on the computer connected to the printer.

2. **Monitor:** Owner opens the Admin Portal.

3. **Automated Fulfillment:** As jobs arrive, the Print Daemon automatically pulls them and sends them to the physical printer.

4. **Verification:** The owner sees the job status shift to "Completed" on the dashboard, bags the printout, and attaches the token number.

Order Lifecycle

Document Upload

↓

Order Creation (Status: Queued)

↓

Print Daemon Polling (Status: Printing)

↓

SumatraPDF Spooling

↓

Physical Printing

↓

Completion (Status: Completed)

SECTION 3: SYSTEM ARCHITECTURE

Campus Print utilizes a decoupled, event-driven architecture designed to bridge public cloud traffic with local, firewalled printer hardware securely.

High-Level Architecture Flow

User (Mobile/Web)

↓ [HTTPS]

Cloudflare Tunnel (Public URL)

↓ [Secure TCP]

Local Backend Node.js Server (Port 3001)

↓ [File System / Local Database]

Print Client Daemon (Node.js Script)

↓ [CLI Commands]

SumatraPDF / Windows Print Spooler

↓ [WSD/USB/Network]

Physical HP Printer

Component Explanations

1. **Frontend (Vite + React + Tailwind):** Serves the UI for both Students and Admins. Built for speed and responsiveness.

2. **Backend (Express + Multer):** Handles file uploads, PDF parsing, job queue management, and SSE broadcasting.

3. **Database (JSON/File-based DB):** A lightweight local persistence layer (`db.json`) used for the MVP to avoid complex database setups at local print shops.

4. **Print Client Daemon:** A background Node.js process that constantly monitors the backend for `queued` jobs, downloads them, and executes CLI commands to physically print them.

5. **Cloudflare Tunnel:** Exposes the locally running Express server to the public internet securely without requiring Port Forwarding or static IPs.

Request & Print Job Flow

1. **Request Flow:** A student POSTs a multipart/form-data request to `/api/jobs`. The server saves the file, creates a DB record, and broadcasts a `new_job` SSE event.

2. **Print Job Flow:** The Print Daemon, listening to the SSE stream, detects the new job. It fetches the file via GET `/uploads/{filename}`. It then spawns a child process invoking `SumatraPDF.exe` with specific print flags (e.g., `fit,monochrome,duplexlong,2x`). The daemon monitors the Windows WMI Print Spooler class to track exact page-by-page progress.

SECTION 4: FRONTEND DEEP DIVE

Frameworks & Libraries Used

- **React 18:** Core UI library for component-based architecture.
- **Vite:** Next-generation frontend tooling for ultra-fast HMR and optimized production builds.
- **Tailwind CSS v4:** Utility-first CSS framework for rapid, responsive, and modern UI styling.
- **Lucide React:** Consistent, lightweight SVG iconography.
- **pdf-lib:** Client-side PDF manipulation and page counting to calculate fares instantly before upload.

Component Hierarchy

\`\`\`

App

├── Header (Navigation)

├── StudentPortal (Main entry)

│ ├── FileUploadArea (Drag & drop zone)

│ ├── FileConfiguration (Copies, Duplex, Range)

│ └── QueueSummaryView (Live SSE tracking)

└── AdminPortal

├── StatCards (Revenue, Jobs)

└── JobTable (Live queue management)

\`\`\`

Major Pages & Interactions

Student Dashboard / Document Upload

Purpose: Allow students to upload files and configure print settings.

User Interactions:

- Dragging and dropping PDF/Image files.
- Selecting specific page ranges (e.g., `1-3, 5`).
- Toggling Simplex/Duplex printing.
- Viewing instant fare estimates (calculated as Pages * Copies * Base Rate).

State Management: Uses React `useState` for form data and `useEffect` for SSE subscriptions.

Order Tracking (QueueSummaryView)

Purpose: Provide transparency into the queue.

User Interactions: Users can click on their recent jobs to see a detailed breakdown of jobs ahead of them and estimated wait times.

State Management: Listens to SSE events (`connected`, `new_job`, `status_update`) to dynamically re-render the queue order and progress bars without polling.

Admin Dashboard

Purpose: Command center for shop operators.

User Interactions:

- View live hardware status (Printer Online/Offline).
- View financial metrics (Revenue calculated from completed jobs).
- Manually fail or delete jobs if physical errors occur (e.g., paper jams).

State Management: Polls `/api/jobs` initially, then stays synchronized via SSE updates from the Print Daemon.

SECTION 5: BACKEND DEEP DIVE

Backend Framework & Server Architecture

The backend is built on **Node.js** using the **Express.js** framework. It acts as the central hub connecting the public frontend (via Cloudflare) to the local database and the internal Print Client Daemon.

The architecture is monolithic but internally segmented into routes, middleware (Multer for file handling), and a persistent SSE (Server-Sent Events) connection pool.

API Structure & Core Endpoints

`POST /api/jobs` (Upload & Order Creation)

- **Purpose:** Accepts multipart/form-data containing the document and print configuration.
- **Input Payload:** `file` (Binary), `studentName`, `studentEmail`, `copies`, `sides`, `pageRange`, `printMode`.
- **Processing:** Uses `multer` to save the file to `/uploads`. Parses PDFs using regex to verify exact page counts. Calculates estimated completion times based on hardware constraints.
- **Output Payload:** JSON object of the created `PrintJob` including the unique Token.
- **Validation & Security:** File size limits (50MB), MIME type checking (PDF, PNG, JPG).

- **Side-effects:** Broadcasts a `new_job` event via SSE to awaken the Print Daemon.

`GET /api/jobs` (Queue Retrieval)

- **Purpose:** Returns the active queue and recent completed jobs.
- **Output Payload:** Array of `PrintJob` objects.

`GET /api/jobs/stream` (Real-Time SSE)

- **Purpose:** Maintains a persistent open HTTP connection to stream server events to clients.
- **Architecture:** Uses `text/event-stream` headers. Maintains an array of `res` objects (`sseClients`). Contains a 15-second keep-alive ping to prevent Cloudflare tunnel connection timeouts.

`POST /api/jobs/:id/status` (Internal Daemon Hook)

- **Purpose:** Used exclusively by the Print Client Daemon to update job states (`printing`, `completed`, `failed`).
- **Input Payload:** `{ status: '...', progressPercent: N }`.
- **Side-effects:** Broadcasts a `status_update` event to all connected UI clients. If status is `completed`, triggers file cleanup to remove the PDF from the server disk.

SECTION 6: DATABASE DESIGN

For the MVP, a localized JSON-based persistence layer (`server/db.json`) is used to minimize deployment complexity at partner print shops.

Data Lifecycle

1. Document uploaded -> New record appended to `jobs` array with `status: 'queued'`.

2. Daemon picks up job -> Record updated to `status: 'printing'`.

3. Physical printer finishes -> Record updated to `status: 'completed'`.

Core Entity: PrintJobs

- `id` (String) - Primary Key, UUID or Timestamp-based hash.
- `token` (String) - Human-readable identifier (e.g., PRNT-542).
- `fileName` (String) - Original uploaded filename.
- `fileSize` (Integer) - Size in bytes.
- `pageCount` (Integer) - Parsed number of pages.
- `copies` (Integer) - Requested copies (1-10).
- `sides` (Enum: single, double) - Simplex or Duplex setting.
- `pageRange` (String) - Specific pages to print.
- `printMode` (Enum: mono, color) - Ink channel requirement.
- `status` (Enum: queued, printing, completed, failed, printer_offline, paper_empty).
- `studentName` (String) - Owner of the document.
- `createdAt` (ISO String) - Timestamp of creation.
- `progressPercent` (Integer) - Real-time physical printing progress.

Note: In future scalable versions, this local JSON structure will map directly to a managed PostgreSQL database schema.

SECTION 7: QR CODE SYSTEM

Purpose and Workflow

The QR code system acts as the secure physical bridge between the digital order and the physical collection of the printed document.

Generation & Validation Flow

1. **Generation:** When a student successfully submits a print job, the frontend UI generates a unique Token (e.g., PRNT-847). In the full production SaaS model, this token is embedded into a secure URL (`https://campusprint.com/verify/PRNT-847`) and rendered as a QR code using a library like `qrcode.react`.

2. **Presentation:** The student receives an on-screen QR code and takes their mobile device to the print shop.

3. **Scanning & Verification:** The store operator scans the QR code using a standard barcode scanner or their smartphone camera.

4. **Validation:** The scan hits the `/api/verify` endpoint. The backend checks if the job ID exists and if the status is `completed`.

5. **Retrieval:** The UI confirms verification, and the operator hands over the physical documents.

Security Considerations

- QR codes must contain a cryptographic signature to prevent users from generating fake QR codes for unpaid jobs.

- Once a QR code is scanned and the document is handed over, the job state transitions to `collected`, invalidating the QR code for future use.

SECTION 8: PAYMENT SYSTEM

Current Architecture vs Future State

Currently, the system is designed to bypass immediate online payment gates to ensure maximum speed and zero friction during the MVP testing phase. Fare estimates are calculated dynamically on the frontend.

Fare Estimation Logic

\`Fare = (Calculated Pages * Copies) * Base Rate (₹3)\`

- For specific page ranges (e.g., `1-3, 5`), the system parses the string, calculates the exact number of physical pages required, and multiplies it by the copy count.

Future Integration (Stripe / UPI)

When scaled, the payment flow will follow an authorization-capture model:

Upload Document

↓

Backend creates Payment Intent (Stripe API) / UPI Intent URI

↓

Student completes transaction on mobile UI

↓

Webhook hits `/api/webhooks/payment`

↓

Backend verifies signature & captures funds

↓

Job status transitions from `payment_pending` to `queued`

↓

Print Daemon begins execution

Security Considerations

- **Webhook Verification:** All incoming payment webhooks must be cryptographically verified using endpoint secrets to prevent spoofing.
- **Refund Flow:** If the Print Daemon reports a fatal hardware error (`paper_empty`, `printer_offline`), the backend will automatically trigger a refund API call to Stripe/Razorpay.

SECTION 9: PRINTING ENGINE

The Printing Engine is the most critical and complex component of the Campus Print ecosystem. It acts as the bridge between cloud-hosted web servers and physical, legacy printer hardware.

Print Client Daemon Architecture

Because web browsers and remote cloud servers cannot natively talk to a local USB or Wi-Fi printer, Campus Print deploys a lightweight background service called the **Print Client Daemon** (`client.cjs`) on the host computer physically connected to the printer.

How Print Jobs are Created and Processed

1. **Event Listener:** The Daemon maintains a persistent SSE connection to `http://localhost:3001/api/jobs/stream`.

2. **Trigger:** When a student uploads a file, the server broadcasts a `new_job` event.

3. **Download:** The Daemon receives the event, fetches the job metadata via REST, and downloads the physical PDF file to a local temporary directory (`C:\temp\campus-print\`).

4. **Command Execution:** The Daemon spawns a child process and uses **SumatraPDF.exe** via Command Line Interface (CLI) to bypass the standard Windows Print Dialog.

- Command Example: `SumatraPDF.exe -print-to "HP Smart Tank" -print-settings "fit,monochrome,duplexlong,2x,1-3" "file.pdf"`

5. **Spooler Monitoring:** Using PowerShell and WMI (Windows Management Instrumentation), the Daemon hooks into the `Win32_PrintJob` class to monitor the exact progress of the job in the Windows Spooler queue.

6. **State Syncing:** As pages physically print, the Daemon sends `POST /api/jobs/{id}/status` updates back to the backend, which in turn broadcasts them to the Student UI.

Printer Status Lifecycle

Pending (Queued in Backend)

↓

Processing (Daemon Downloading & Spooling)

↓

Printing (Job inside Windows Spooler, tracking pages)

↓

Completed (Job exits Spooler) OR Failed (Spooler reports error/offline)

Hardware Integration Constraints

- To ensure reliability, the engine uses SumatraPDF instead of native PowerShell `Start-Process -Verb Print`, as native Windows handlers often fail to respect duplex and specific copy counts via command line.
- The daemon continuously polls `Win32_Printer` to detect if the physical hardware is offline or out of paper, pausing the queue automatically if errors are detected.

SECTION 10: DATA FLOW ANALYSIS

Workflow 1: Document Upload to Print

1. **Frontend Validation:** User drops a PDF. `pdf-lib` extracts page counts locally. Form validates inputs (copies max 10, monochrome locked).

2. **Backend Upload:** Multipart data posted. Multer intercepts the binary stream and writes to `/uploads/TIMESTAMP-filename.pdf`.

3. **Storage:** File is physically saved on the local drive of the host machine.

4. **Order Creation:** Backend constructs a `PrintJob` object, generates a token, appends to `db.json`.

5. **SSE Broadcast:** Server flushes `data: {"type":"new_job", "jobId":"..."}` to the open event stream.

6. **Daemon Execution:** Print Client receives SSE, fetches file from `/uploads`, issues SumatraPDF CLI command.

7. **Cleanup:** Once printed, Daemon notifies Backend. Backend executes `fs.unlinkSync()` to permanently delete the document from the hard drive, ensuring data privacy.

SECTION 11: SECURITY REVIEW

Security Vectors & Mitigations

1. File Upload Security

- **Vulnerability:** Malicious users uploading executable files (.exe, .sh) or overly large files to crash the server.
- **Mitigation:** Multer is strictly configured to limit file sizes to 50MB. MIME type validation ensures only `application/pdf` and standard image formats are accepted. Files are renamed using cryptographic timestamps to prevent directory traversal attacks.

2. Document Privacy

- **Vulnerability:** Sensitive student documents (resumes, ID cards, assignments) left exposed on the print shop's computer.
- **Mitigation:** The system implements an aggressive ephemeral storage model. The moment the Print Daemon verifies the job has successfully cleared the Windows Print Spooler, it sends a webhook to the backend. The backend immediately executes a hard deletion (`fs.unlinkSync`) of the file.

3. Tunnel Exposure

- **Vulnerability:** Exposing a local server to the public internet via Cloudflare Tunnel.
- **Mitigation:** Cloudflare automatically provides DDoS protection and SSL/TLS encryption. The backend does not expose any administrative mutation routes without verification.

4. Queue Manipulation

- **Vulnerability:** Students spamming the upload endpoint to flood the printer.
- **Mitigation:** Future iterations require rate-limiting by IP address and requiring payment intent verification *before* a job is injected into the physical print queue.

SECTION 12: DEPLOYMENT GUIDE

The system is designed to be deployed with zero financial cost, utilizing existing hardware and free tier services.

Local Development & MVP Deployment Setup

Step 1: Host Computer Setup

1. Identify the computer physically connected (via USB or Wi-Fi) to the target printer (e.g., HP Smart Tank).

2. Install **Node.js** (v18+) and **Git**.

3. Clone or copy the Campus Print repository to this machine.

4. Run `npm install` in the root directory.

Step 2: Configuration

1. Open Windows PowerShell and run: `Get-Printer | Select-Object Name`

2. Copy the exact name of the target printer.

3. Edit `print-client/config.json`:

- Set `mockPrinter: false`
- Set `printerName: "Exact Printer Name Here"`

Step 3: Execution

Open three separate terminal windows and execute:

1. **Terminal 1 (Backend):** `npm run server`
2. **Terminal 2 (Frontend):** `npm run dev -- --host 0.0.0.0`
3. **Terminal 3 (Print Daemon):** `node print-client/client.cjs`

Step 4: Public Internet Tunneling

To allow students to access the portal from their phones without being on the same local network, use Cloudflare Tunnels:

1. Download `cloudflared.exe`.
2. Run `cloudflared tunnel --url http://localhost:3000`
3. Share the generated `.trycloudflare.com` URL with students.

Hosting Requirements

- No cloud servers (AWS/DigitalOcean) are required for the MVP. The entire stack runs on the Print Shop's local hardware, ensuring zero latency between the server and the printer.

SECTION 13: PERFORMANCE & SCALABILITY

Current Limitations & Benchmarks

In its current MVP state, the system is highly optimized for a single-store, localized deployment.

- **Database:** The `db.json` implementation limits concurrent write operations. Node.js `fs.writeFileSync` is synchronous, meaning high concurrency (e.g., 50+ students uploading simultaneously) will cause event loop blocking.
- **Print Spooler:** Legacy physical printers (like the HP Smart Tank) print at a hardware-capped speed of 10-15 pages per minute. The primary bottleneck is the physical hardware, not the software.
- **Network:** Cloudflare Quick Tunnels drop idle connections, which we mitigated using SSE Keep-Alive pings.

Scaling Roadmap

To scale from a single shop to a campus-wide or multi-campus SaaS platform:

1. **Database Migration:** Replace `db.json` with **PostgreSQL** to handle transactional integrity, relational mapping between Shops, Users, and Orders, and highly concurrent writes.

2. **Object Storage:** Migrate local `/uploads` to **AWS S3** or **Cloudflare R2**. This decouples the web server from the physical storage, allowing horizontal scaling of the Express APIs.

3. **Message Broker:** Replace simple SSE arrays with **Redis Pub/Sub** or **RabbitMQ**. This guarantees message delivery to Print Daemons even if

network connections drop briefly.

4. **Daemon Authentication:** Implement JWT or API Key authentication for the Print Client Daemons so multiple shops can securely connect to a centralized cloud backend without pulling each other's jobs.

SECTION 14: INTERVIEW PREPARATION

Use these answers to confidently explain the project during technical interviews or pitches.

1. "Explain your project and what problem it solves."

Elevator Pitch: "Campus Print is a decentralized, real-time print management system designed to eliminate physical queues at university print shops. Traditionally, students waste time transferring files via USBs or public PCs. I built a platform where students upload documents via a web app, and the system automatically calculates fares and routes the job directly to the shop's physical printer using a custom background daemon. It turns a manual, 5-minute ordeal into a 10-second frictionless experience."

2. "What technologies did you use and why?"

Technical Answer: "I used a modern JavaScript stack. The frontend is built with React, Vite, and Tailwind CSS for rapid, responsive UI development. The backend is Node.js and Express. The most interesting part is the Print Daemon—I wrote a custom Node script that uses Server-Sent Events (SSE) to listen for cloud updates, then uses the `child_process` module to spawn SumatraPDF CLI commands, hooking directly into the Windows Print Spooler via WMI to track physical page-by-page progress."

3. "What was the biggest technical challenge you faced?"

Advanced Answer: "Bridging the gap between the public web and a local, firewalled legacy printer. Cloud servers can't just 'ping' a local USB printer. My solution was the Print Client Daemon pattern. I had to ensure the Daemon maintained a persistent connection to the server without timing out. I used SSE with keep-alive pings. The second challenge was Windows Spooler unreliability—native PowerShell commands often ignored duplex or copy

settings. I resolved this by integrating SumatraPDF for deterministic command-line print spooling."

4. "How would you scale this for 100 universities?"

Scalability Answer: "I would transition the architecture to a multi-tenant SaaS model. I'd replace the local file database with PostgreSQL and move document storage to AWS S3. I'd implement Redis for queue management and Pub/Sub to route jobs to specific shop Daemons. Each shop would install the Daemon, authenticate via an API key, and subscribe only to their specific shop's channel."

SECTION 15: COMPLETE LEARNING GUIDE

To fully master the concepts in this project, here is a breakdown of the core technologies utilized:

1. Server-Sent Events (SSE)

- **What it is:** A standard allowing a web server to push real-time updates to a client over a single, long-lived HTTP connection.
- **Why used:** Unlike WebSockets (which are bidirectional), we only need unidirectional data (Server -> Client/Daemon) to say "A new job arrived" or "Printing is at 50%". SSE is lighter and runs over standard HTTP/HTTPS.

2. Windows Management Instrumentation (WMI)

- **What it is:** The infrastructure for management data and operations on Windows-based operating systems.
- **How it works here:** The Print Daemon executes PowerShell queries against the `Win32_PrintJob` and `Win32_Printer` WMI classes to read the physical spooler status, detecting if the printer is offline or how many pages have successfully printed.

3. CLI Spooling (SumatraPDF)

- **What it is:** SumatraPDF is an open-source PDF viewer known for its robust command-line interface.
- **Why used:** Native Windows print handlers abstract away deep settings like duplexing or collated copies when invoked via script. SumatraPDF accepts explicit flags (`duplexlong`, `2x`), guaranteeing the physical printer respects the user's web configurations.

SECTION 16: CODE REVIEW & ARCHITECTURE ASSESSMENT

Architectural Strengths

1. **Decoupled Daemon Pattern:** By separating the web server from the physical hardware controller, the architecture successfully bridges cloud/local environments securely.

2. **Zero-Friction UX:** Removing the payment/login gates for the MVP ensures absolute minimum time-to-value for the end user.

3. **Data Privacy:** Implementing immediate `fs.unlinkSync()` after print completion enforces a strong data retention policy, highly valuable in academic environments.

Technical Debt & Weaknesses

1. **Concurrency Issues in DB:** The use of synchronous `fs.readFileSync` and `fs.writeFileSync` in `server/db.ts` will block the Node.js event loop under heavy load.

2. **In-Memory SSE Arrays:** The `sseClients` array lives in the Node.js process memory. If the Express server restarts or scales to multiple instances, SSE connections will drop and fail to broadcast across instances.

Recommendations for Future Iterations

1. **Immediate fix:** Wrap database read/writes in asynchronous `fs.promises` to prevent event loop blocking.

2. **Medium-term:** Implement a CRON job to sweep the `/uploads` directory nightly for orphaned files (files that failed to print and thus bypassed the deletion logic).

3. **Long-term:** Containerize the backend and deploy to a managed cloud provider, keeping only the lightweight Daemon script on the local print shop hardware.